

Fundamentos de Spring Boot - Apuntes

Spring es un framework para aplicaciones empresariales.

Las aplicaciones empresariales deben ser escalables, modulares, integrales, seguras, disponibles y de multiprocesos.

- **INVERSIÓN DE CONTROL**

Es un diseño del software que trata de darle la vuelta a quién controla la ejecución, es decir, la responsabilidad del flujo de la app no la tiene el programador, se delega ese control.

Se puede llevar a cabo a través de inyecciones de dependencias, de callbacks, de Service Locators, etc

→ **Beneficios de la ioc:**

- Desacoplamiento: podemos crear piezas de software poco dependientes de otras.
- Facilidad para las pruebas: al no depender de otras piezas, podemos hacer simulaciones modularmente, haciendo mocks, es decir, dobles.
- Flexibilidad: podemos declarar una dependencia como tipo interfaz. (ej; en vez de usar para una lista ArrayList, podemos usar una interfaz, y al usarla podemos elegir entre usar ArrayList, LinkedList, etc). Cambiar la implementación concreta no implica cambiar el code del lugar en el que está declarada esa dependencia.

- **INYECCIÓN DE DEPENDENCIAS**

Es una de las formas de aplicar la inversión de control en el code, se encarga de hacerlo entre los distintos objetos que tenemos en el code de forma eficiente.

En lugar de que las clases gestionen las dependencias, proporcionamos esas dependencias desde fuera, se inyectan de forma externa.

Esta inyección lo suele hacer un framework o un contenedor de inversión de control.

Cuando una clase declara sus dependencias, es como si nosotros estuviéramos declarando la lista de la compra antes de ir a comprar

→ **Beneficios de la inyección de dependencias:**

- Clase no responsable: las clases no son las responsables de crear las instancias de los objetos de los que dependen de ella. (Yo necesito una serie de objetos pero yo no los creo, alguien externo lo hace).

Ej:

Sin inyección de dependencias:

```
public class Car {  
    private Engine engine;  
  
    public Car() {  
        this.engine = new Engine(); // Car crea su propia dependencia  
    }  
}
```

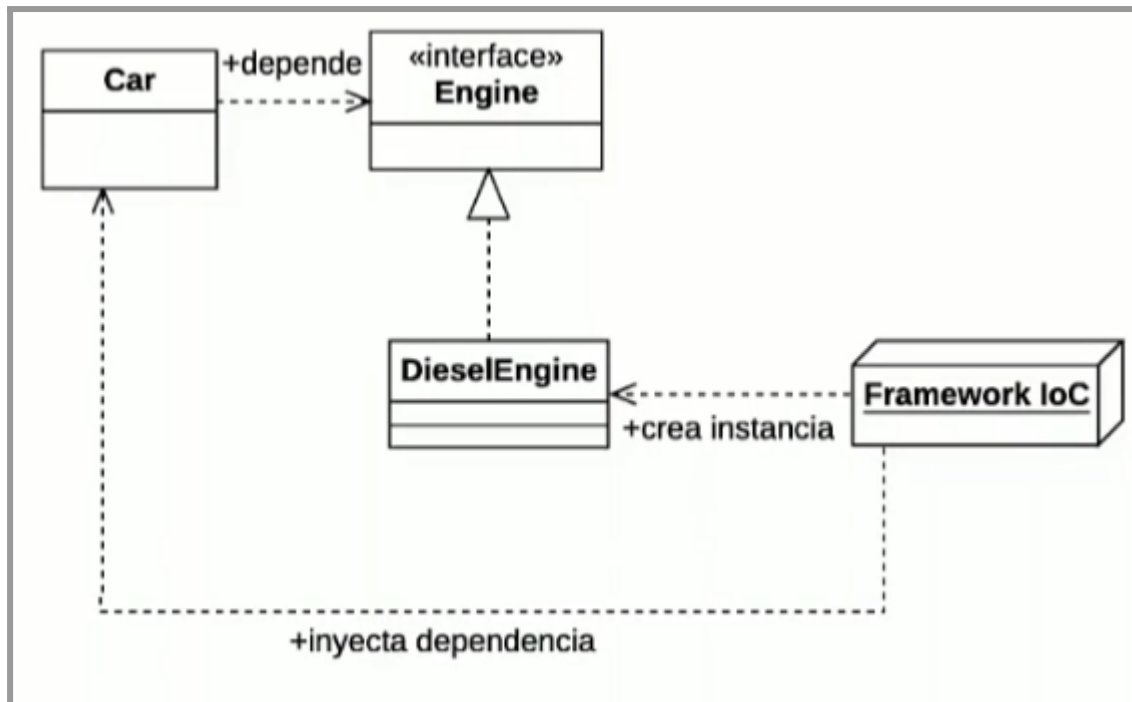
VS

Con inyección de dependencias:

```
public class Car {  
    private Engine engine;  
  
    // Inyección de dependencias a través del constructor  
    public Car(Engine engine) {  
        this.engine = engine;  
    }  
}
```

En este caso se hace de forma externa, se le proporciona el motor por parámetros. La clase coche declara que necesita un motor, pues lo dice por sus campos o atributos y, en este caso, por los parámetros del constructor se le pasa la dependencia que necesita, es decir, el motor.

Si esa inyección de dependencia del motor a la clase coche la hiciéramos con una interfaz, sería mucho más potente porque no la “limitamos” a un tipo en concreto, lo hacemos de forma “abstracta”, sin terminar de definir nada:



De esta forma se podría poner a la clase coche motores de diferentes tipos, gasolina, diesel, eléctricos o híbridos.

Todo esto lo configuraríamos sin cambiar el code de la clase coche, ese es el potencial de todo esto.

La instancia como tal la crea el framework de loc, no la crea la clase coche (es decir, nosotros mediante code) pues ella no es la que hace la tarea, la delega.

Posteriormente, ve las dependencias que hay entre los objetos y ve que coche necesita un motor, concretamente en este ejemplo, un motor de tipo diesel.

- Las mismas que las de IOC:
 - + desacoplamiento
 - + facilidad para el testing, con dobles.
 - + flexibilidad con el uso de interfaces
- Precisión: si comparamos otras formas de inversión de control, podemos ver que la inyección de dependencias es mucho más fina, en el sentido de que puede dar exactamente lo que necesita cada objeto, no es como los service locators, que son como contenedores.

SPRING BOOT

Es un framework, al igual que lo es spring, sin embargo, este es mucho más potente dado que automatiza muchas cosas y evita que tengamos que estar “haciendo el mismo code” a lo largo del desarrollo de aplicaciones. Es un framework construido como base usando spring, de ahí que una parte de su nombre sea ‘Spring’.

Concretamente es un framework opinionado, es decir, en opinión de sus creadores, es la mejor manera de crear una aplicación de spring.

Usa convención sobre configuración, es un patrón que minimiza la toma de decisiones triviales que el programador tiene que tomar, nos da las decisiones tomadas, solo tenemos que dar las decisiones diferentes como programadores. (Ej: por el code que tenemos hecho, toma la decisión de llamar a una lista ‘facturas’ pero nosotros queremos hacerlo algo distinto por cualquier motivo, pues tendríamos que cambiar el name a, por ejemplo, facturasClientes).

Hace de forma automática la estructura de proyectos, ficheros xml, beans múltiples, etc.

Summary:

- Aplicaciones autónomas
- Servidor incrustado
- Dependencias predefinidas
- Configuración automática

Características:

- Aplicaciones listas para producción
- Spring Initializr (asistente)
- Todo de forma interna, sin código andamio ni nada por el estilo.

→ Podemos hacer con spring boot todo tipo de aplicaciones java, es decir, aplicaciones que funcionan desde el terminal, apps webs, api rest, microservicios, etc.

Beneficios:

- Rapidez en el desarrollo
- Menos código repetitivo/plantilla
- Facilidad para el despliegue
- Escalabilidad

MAEVEN VS GRADLE

Son gestores de proyectos que se encargan pues de lo que indica su nombre, de gestionar proyectos, es decir, sus librerías, empaquetación, despliegue.

Maven funciona con una estructura predefinida, con el uso del fichero pom.xml, también se caracteriza por los ciclos de vida de fases y por el manejo de dependencias,.

Ventajas:

- Estandarizado, muy usado
- Convención sobre configuración
- Gran cantidad de plugins
- Soporte para Spring Boot

Desventajas:

- Menor flexibilidad
- Verboso
- Más lento
- Soporte para Spring Boot

Por el otro lado, tenemos que Gradle tiene un mejor rendimiento, es más flexible, tiene una sintaxis más limpia y es extensible. Aunque su curva de aprendizaje es mayor, está menos estandarizado y tenemos mejores ejemplos.

SPRING INITIALIZR

Nosotros no vamos a usarlo, pero se puede probar:

<https://start.spring.io/>

ESTRUCTURA PROYECTO MAVEN

La estructura suele ser generalmente de la siguiente forma:

```
mi-proyecto/
├── src/
│   ├── main/
│   │   ├── java/           -> Código fuente
│   │   ├── resources/      -> Archivos de recursos (configuraciones, etc.)
│   └── test/
│       ├── java/           -> Pruebas unitarias
│       └── resources/      -> Archivos de recursos para pruebas
└── pom.xml                 -> Archivo de configuración de Maven
```

En primer lugar tenemos que en el main, encontramos el tag:

@SpringBootApplication

→ Casi nunca vamos a tocar esta clase, la main default.

INYECCIÓN DE DEPENDENCIAS

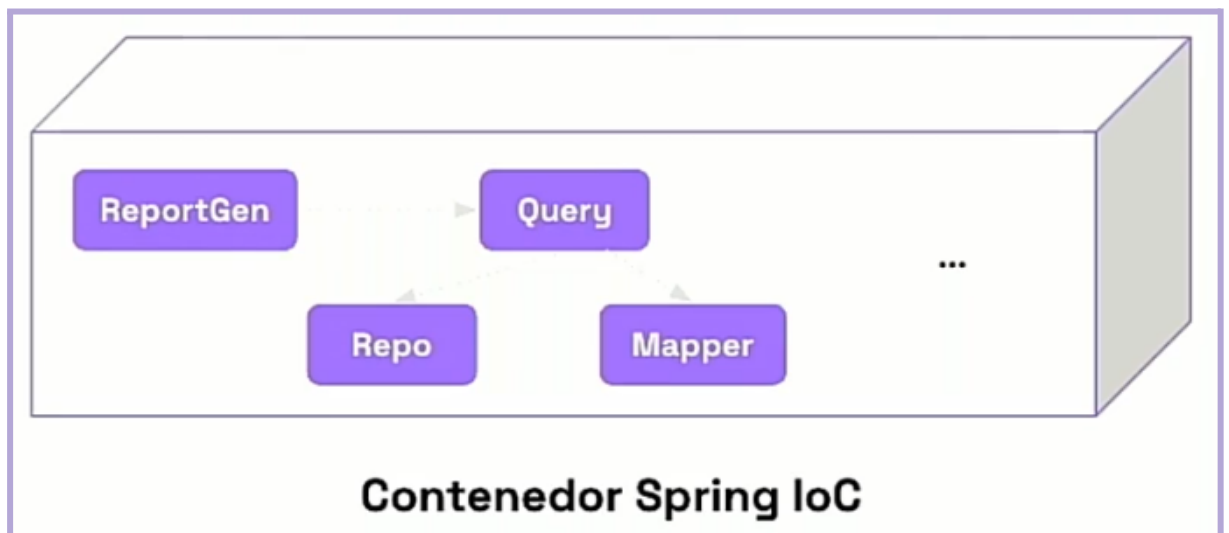
- ¿Qué es un bean?

Es una clase java normal y corriente (POJO) gestionada por el contenedor de inversión de control.

Si sumamos a nuestra clase POJO java unos metadatos de configuración y los metemos en un contenedor de inversión de control (que es spring), obtenemos un bean

- ¿Cómo funciona la inyección de dependencias entre beans?

Transformamos nuestras clases en beans, con las configuraciones de metadatos y de esta forma articulamos la inyección de dependencias entre los beans que usamos. Ej:



- ¿Cómo se transforma una clase java POJO en un bean?

Se puede hacer de varias formas.

La primera de ellas es a través de xml, que es la forma legacy, es decir, la antigua.

La segunda forma es con anotaciones @algo

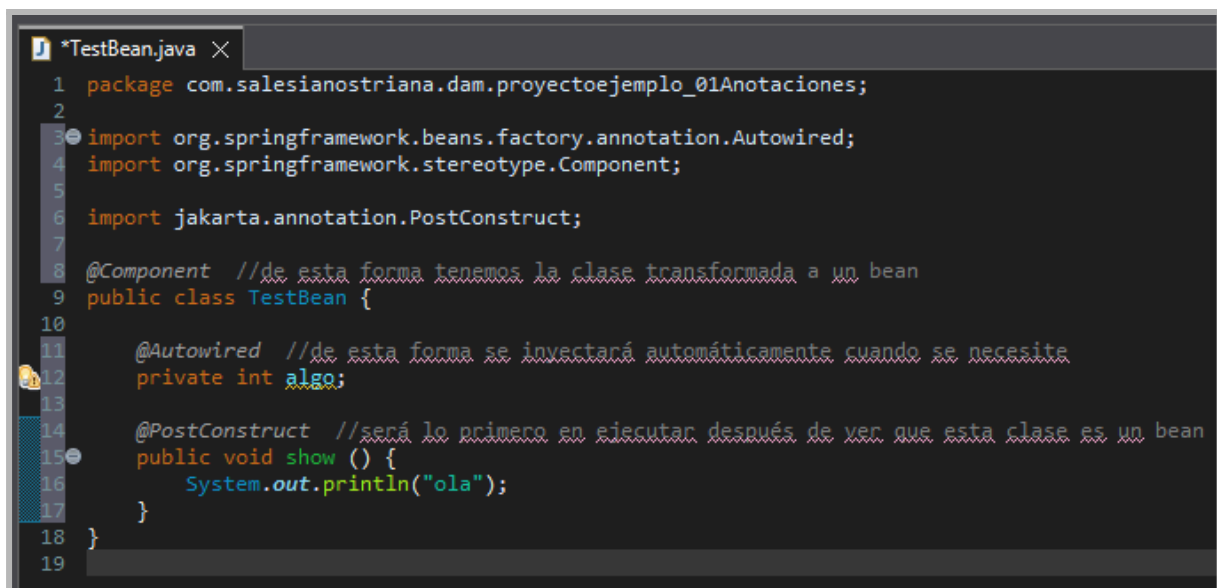
La tercera es a través de JavaConfig

Nosotros lo vamos a hacer a través de anotaciones pues es sencilla, aunque podemos hacerlo también con JavaConfig y también veremos como hacerlo de esa forma.

→ [Anotaciones](#)

- Son metadatos sobre el code
 - `@algo`
 - Se pueden aplicar a nivel de clase, atributos, métodos, etc.
-
- + **`@Component`** → para transformar la clase en un bean
 - + **`@PostConstruct`** → para que cuando se ejecute, si se ha detectado la clase como un bean (con `@Component`), lo que va a hacer acto seguido es ejecutar lo que esté anotado como `@PostConstruct`
 - + **`@Autowired`** → para hacer la inyección de dependencias de forma automática

Ej:



```
1 package com.salesianostriana.dam.proyecto ejemplo_01Anotaciones;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.stereotype.Component;
5
6 import jakarta.annotation.PostConstruct;
7
8 @Component //de esta forma tenemos la clase transformada a un bean
9 public class TestBean {
10
11     @Autowired //de esta forma se inyectará automáticamente cuando se necesita
12     private int algo;
13
14     @PostConstruct //será lo primero en ejecutar después de ver que esta clase es un bean
15     public void show () {
16         System.out.println("ola");
17     }
18 }
19
```

→ [JavaConfig](#)

- Como ya hemos visto anteriormente, es otra forma de transformar clases a beans
 - **LA DIFF, ANOTACIONES VS JAVACONFIG:** cuando lo hacemos con anotaciones, estas van en las clases que queremos que sean beans, con javaconfig, las anotaciones van en una clase externa, que también es un bean
 - **Ventajas de javaconfig:** podemos importar una clase con code ajeno y sin tener que acceder a su code y modificarlo, podemos transformarlo en un bean de forma externa
 - **El uso de anotaciones y de javaconfig pueden coexistir y funcionar correctamente en un solo proyecto**
-
- + **`@Configuration`** → para indicar que una clase va a crear beans. (Que va a tener beans en su interior)

- + **@Bean** → invoca un method para crear un bean una sola vez y lo aloja en el contenedor de inversión de control.

Ej:

```

TestBean.java  TestBeanV2.java  ConfigBeans.java X
1 package com.salesianostriana.dam.proyectoejemplo_01JavaConfig;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5
6 @Configuration
7 public class ConfigBeans {
8
9     //creamos un method que devuelve una instancia de la clase que queremos transformar en un bean
10    //el name del method puede ser cualquiera pero hay que tener en cuenta que este name va a ser el que va a tener el bean
11    //(generalmente, para ahorrar problemas, le vamos a dar el mismo nombre)
12    @Bean
13    public TestBeanV2 otroBean () {
14        return new TestBeanV2();
15    }
16 }
17

```

//creamos un method que devuelve una instancia de la clase que queremos transformar en un bean
 //el name del method puede ser cualquiera pero hay que tener en cuenta que este name va a ser el que va a tener el bean
 //(generalmente, para ahorrar problemas, le vamos a dar el mismo nombre)

- ¿Cómo hacen las inyecciones de dependencias de forma automática?
 Se puede hacer de varias formas.

La primera de ellas es manualmente, usando XML, DSL Groovy, JavaConfig

La segunda forma es automáticamente, usando anotaciones

El enfoque que vamos a usar es siempre que podamos anotaciones y las ocasiones en las que no podamos, haremos la inyección de dependencias de forma manual con JavaConfig

→ Inyección automática:

- Busca dentro del contenedor de inversión de control al candidato perfecto para satisfacer la dependencia que tengo.
- Para hacer la inyección automática, usamos la anotación

@Autowired:

- + Para variables de instancia (atributos)
- + Para constructores
- + Para métodos (setter)

- **Ventajas:**

- + Simplicidad y reducción de code

- **Desventaja:**

- + Menos claridad en pruebas unitarias

Ej: para simplificar, lo voy a explicar en lugar de poner varias capturas por cada clase.

Si tenemos una clase llamada Repositorio y tenemos otra clase llamada Servicio en cuyos atributos se encuentra uno de tipo Repositorio, debemos poner el el atb de Servicio en y en Repositorio @Autowired para que se haga solo en Servicio. Si seguimos con el ejemplo, si tenemos en el main de la app que necesitamos usar un objeto de tipo Servicio, pues tenemos que tener tanto en Servicio como en el main @Autowired para que se de la inyección automática. De esta forma tendremos las inyecciones automáticas desde repositorio en servicio y desde servicio en main.

- **@Autowired en constructor:**
 - + No es obligatorio si hay un único constructor
- **Ventajas:**
 - + Inmutabilidad, fácil de probar, mejor legibilidad
- **Desventaja:**
 - + Code adicional

Ej: para simplificar, lo voy a explicar en lugar de poner varias capturas por cada clase.

- **@Autowired en métodos setters:**
 - **Ventajas:**
 - + Flexibilidad, pruebas
 - **Desventaja:**
 - + Mayor riesgo de inconsistencia

→ También se puede poner en cualquier tipo de método, no solo en un setter.
→ OJO: no inyectar la misma dependencia dos veces
→ La inyección automática tmb se puede hacer opcional, se hace con @Autowired(required = false), cuidado que si lo necesitamos y usamos esto, puede dar NullPointerException

- **@Autowired en clases que implementan una interfaz:**

Se puede hacer aplicando cualquiera de las estrategias anteriores

→ OJO: las interfaces pueden indicar el tipo de dependencias pero las interfaces nunca son un bean

- **@Autowired en múltiples beans del mismo tipo en un array o colección:**
 - **Ventajas:**
 - + Flexibilidad, pruebas
 - **Desventaja:**
 - + Mayor riesgo de inconsistencia

→ No es lo más habitual aunque el framework lo usa mucho internamente

SUMMARY:

- Inyección manual:
 - + control total
 - + lógica de inicialización
 - + condiciones
- Inyección automática:
 - + dependencias simples y claras
 - + simplificar
 - + spring boot + anotaciones

→ default way: anotaciones > javaconfig > ...
(autowired) > config java

→ autowired en variables, constructores, se puede hacer en clases e interfaces y también tenemos inyección opcionales y sobre arrays y colecciones.

ANOTACIONES

@Primary:

Se puede usar cuando un bean declara una dependencia donde el tipo de referencia sea una interfaz y tenemos más de un bean que implementan esa interfaz.

Con esta anotación haremos que uno de los beans, aunque sea del mismo tipo que otro, destaque sobre este.

→ SOLO PUEDE HABER UN @PRIMARY POR TIPO

→ Esta anotación se puede usar a nivel de clase o junto a la notación @Bean si estamos usando javaconfig

@Qualifier:

Ya hemos visto que si tenemos dos anotaciones referidas a un mismo bean, podemos usar @primary para que esa sea la principal, pero, ¿y si quiero utilizar otro que no esté anotado con @Primary? Es decir, @Primary siempre salvo en este caso que necesito x

Para ello tenemos la anotación @Qualifier, que con el uso de @Autowired nos va a permitir afinar la inyección automática. Es decir, quiero un bean de este tipo y en realidad quiero ese concretamente

→ PARA USAR @Qualifier + @Autowired:

@Qualifier("nombreDelBean")

+ El nombre del bean suele ser el name de la clase (pero con la primera letra en minúsculas), el nombre del método (si estamos usando javaconfig)

→ Esta anotación se puede usar a nivel de clase o junto a la notación @Bean si estamos usando javaconfig

TIP: uso de @Autowired + @Primary + @Qualifier para afinar la inyección de dependencias

@Value (anotación de Spring, no confundir con la de Lombok)

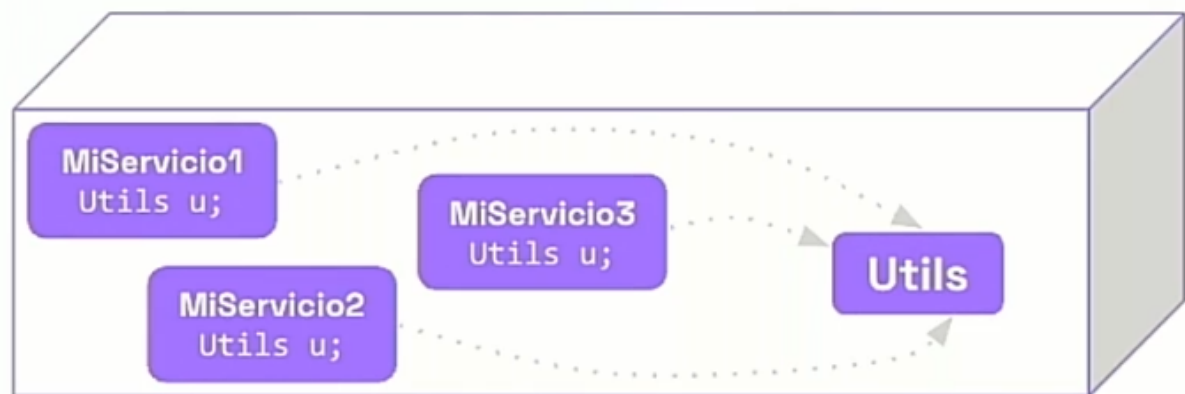
Hasta ahora no hemos podido hacer inyección de dependencias de tipos primitivos ni de String.

Se hace con @Value + application.properties

@Scope

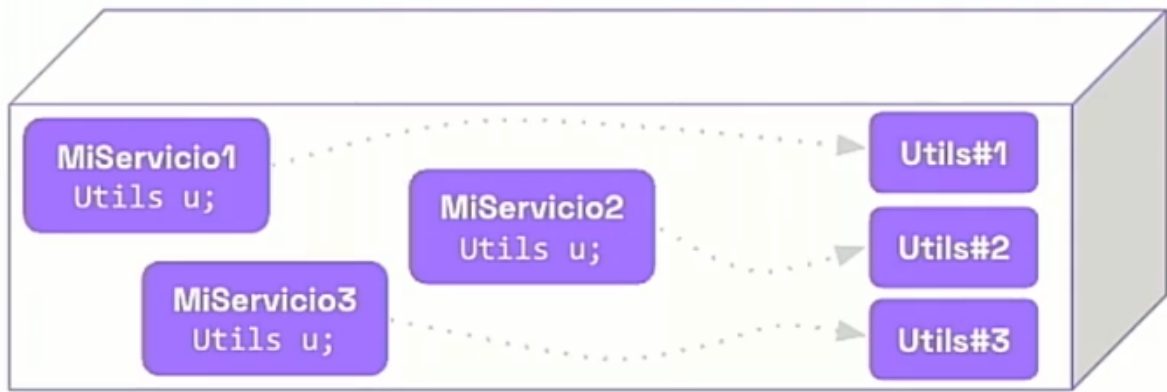
Es el ámbito, es decir, el periodo mediante el cual está viva y podemos acceder a ella.

Default es singleton, solo se hace una vez, lo que se hace es pasar la dirección de memoria donde está la instancia.



```
com.openwebinars.demo.scope.Utils@72e34f77
com.openwebinars.demo.scope.Utils@72e34f77
com.openwebinars.demo.scope.Utils@72e34f77
```

Por otro lado, tenemos Prototype, que crea una nueva instancia por cada vez que se demanda, es lo contrario a la opción de scope default. Cabe mencionar que la destrucción de la instancia no se hace automáticamente tras ser usada, por lo que tendremos que hacerla nosotros como programadores.

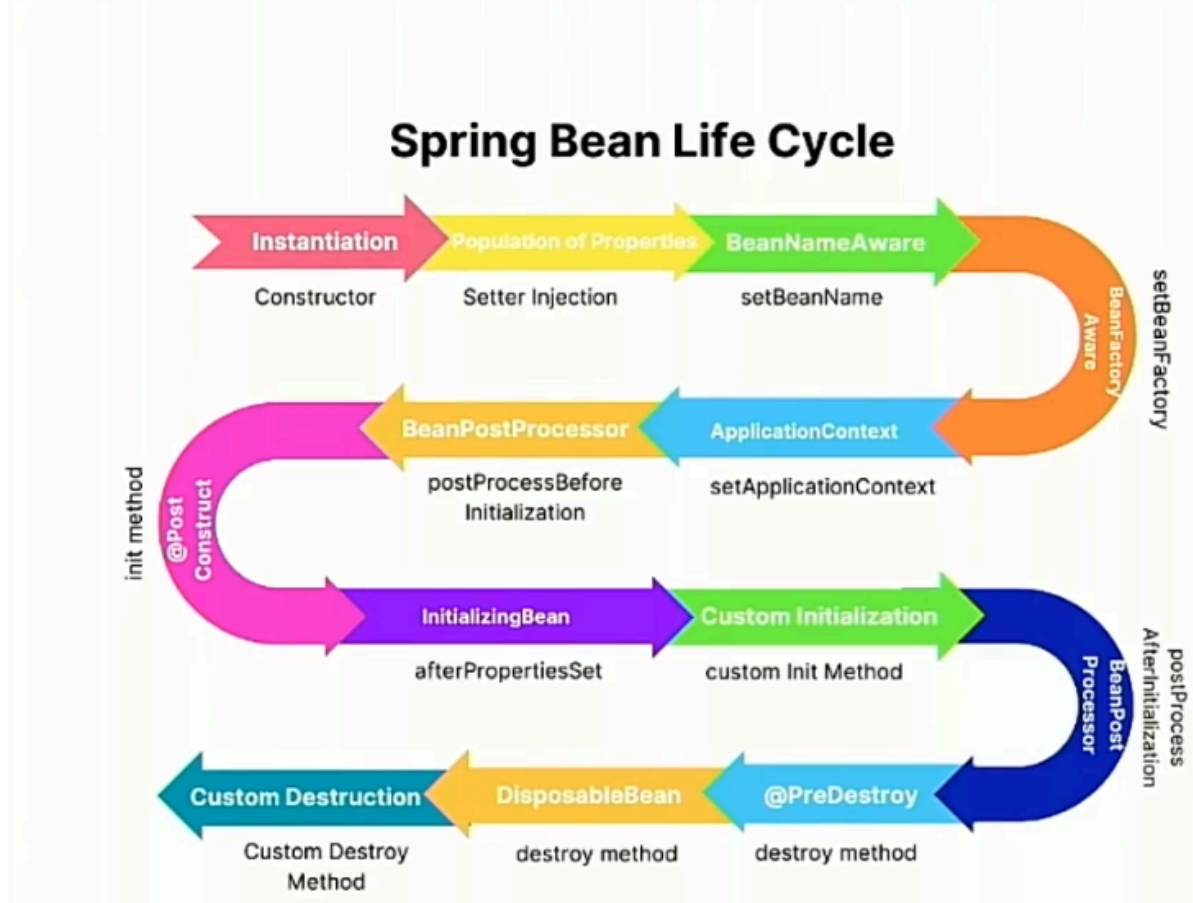


```
com.openwebinars.demo.scope.Utils@c65a5ef  
com.openwebinars.demo.scope.Utils@6b5176f2  
com.openwebinars.demo.scope.Utils@b672aa8
```

También tenemos ámbitos de otros tipos, como los request con http, @sesion para que cada sesión de http al hacer login se guarden elementos y al hacer log out que se borre. También tenemos application, que es como singleton, crea una instancia del bean pero para toda la app web entera. WebSocket es como prototype pero al cerrar en la web.

@Postconstruct & @Predestroy

CICLO DE VIDA DE UN BEAN



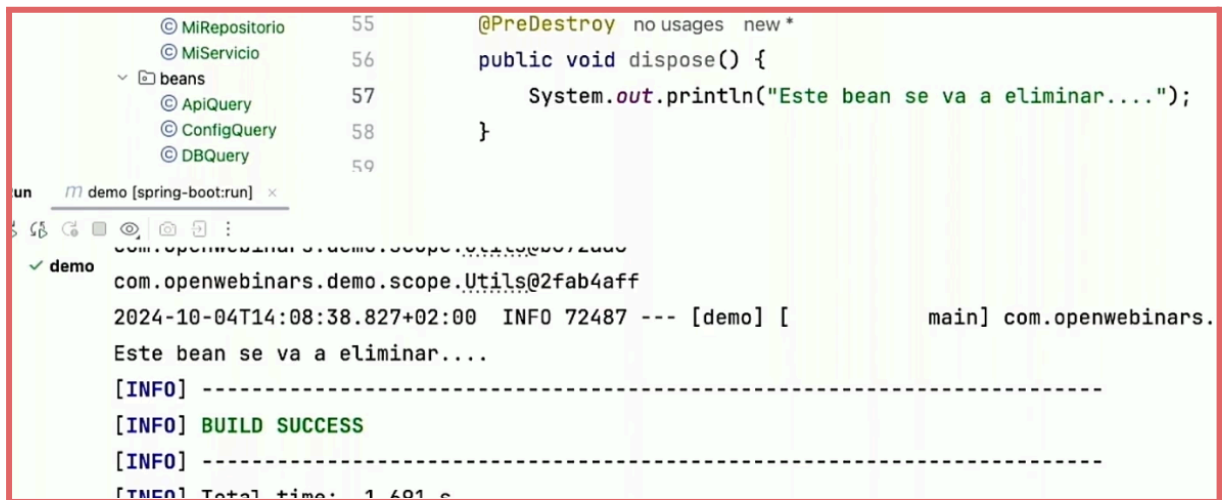
Primero se crea el objeto pero no está configurado, después se inyectan las dependencias, después inicializa las propiedades, invoca métodos talkbacks.

Pues estas anotaciones son para hacer cosas con el bean, por ejemplo, antes de destruirse.

→ **@Postconstruct** se ejecuta cuando el bean está listo para usarse. En este punto las dependencias están resueltas o ya habrían dado fallo

→ **@Predestoy** se ejecuta cuando el bean se está preparando para destruirse, es decir, se ejecuta antes de destruirse.

Ej:



The screenshot shows an IDE with a project named 'demo' (spring-boot:run). The left sidebar shows a package structure: 'beans' containing 'ApiQuery', 'ConfigQuery', and 'DBQuery'. The main editor shows the following Java code:

```
55 @PreDestroy no usages new *
56 public void dispose() {
57     System.out.println("Este bean se va a eliminar....");
58 }
59
```

The console output at the bottom shows the following log messages:

```
com.openwebinars.demo.scope.Utils@2fab4aff
2024-10-04T14:08:38.827+02:00 INFO 72487 --- [demo] [main] com.openwebinars.
Este bean se va a eliminar....
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.491 s
```

Estereotipos

Antes de empezar con los estereotipos, es importante hablar sobre el escaneo de componentes. Es una facultad que tienen las apps de spring, que sirve para encontrar los beans que están anotados por el code.

- Por defecto, Spring Boot lo trae activado (el escaneo de componentes)
- Lo que hace es buscar beans desde el paquete raíz

Existen 4 tipos de estereotipos (a nivel de clase):

- @Component
- @Service
- @Repository
- @Controller

>>> @Component:

- Es la más genérica, las demás son derivadas de esta
- Útil para cualquier cosa

<<< @Service:

- Capa de servicio
- Lógica de negocio
- Semántica más clara

(Es igual que component, pero es para que sea más legible para nosotros)

<<< @Repository:

--Capa de acceso a datos (DAO)

--Manejo de excepciones

--Fuentes de datos

(Este tipo no solo es semántica, puede ayudarnos a darle funcionalidad a las apps, ej: todas las excepciones que se manejen aquí son específicas de acceso a datos, o configurar una relación a una bbdd en particular, etc. Es un bean que está orientado al acceso a los datos)

<<< @Controller:

--Capa de controlador (MVC)

--Gestión de peticiones HTTP

--@GetMapping, @PostMapping

(Recibe peticiones y produce respuestas, ej: en app web modelo vista controlador, va a ser capaz de recibir una petición tipo http y también va a ser capaz de producir otra respuesta). Se usa para conectar endpoints y demás.

→ Beneficios de usar los estereotipos:

- + Claridad semántica
- + Component scanning
- + Consistencia de arquitectura
- + Facilidad en el manejo de excepciones